

Algoritmos y Estructuras de Datos

Problem Sets UT2 – Estructuras Jerárquicas.

Fecha de entrega: 29 de abril 2026 23:59.

Se debe subir un archivo zip con las siguientes características:

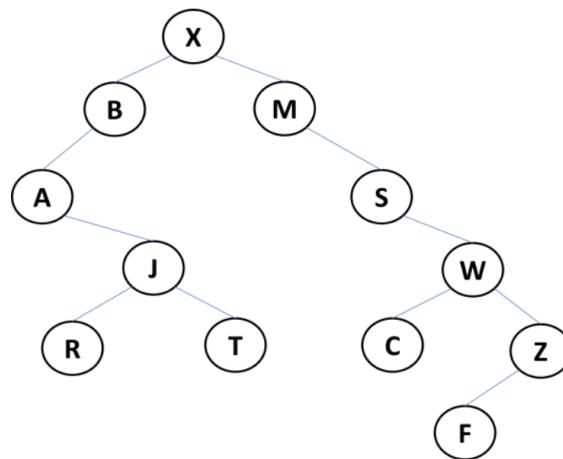
Nombre del archivo: `equipo-{numero de equipo}-{nombre de docente}.zip`

El mismo debe contener todo el código generado e imágenes o documentos en pdf en aquellos ejercicios que no se resuelven en código.

Ejercicio 1

Parte 1

Hallar los recorridos en preorden, postorden e inorden del siguiente árbol binario:



- El árbol binario presentado:
 1. Tiene altura 6.
 2. Cumple con la propiedad de árbol de búsqueda.
 3. Tiene cuatro nodos externos.
 4. Cumple con las propiedades del árbol binario completo.
- Determine la opción correcta:
 1. El postorden es R,T,J,Z,A,C,F,W,B,S,M,X.
 2. El inorden es A,R,J,T,B,X,M,S,C,W,F,Z.
 3. El preorden es X,B,A,S,R,T,M,C,W,J,Z,F.
 4. Ninguna es correcta.
- Determine la opción correcta:

1. En el postorden, “B” aparece antes que “W” porque es un ancestro de ese nodo.
2. En el inorden, “R” aparece después que “A” porque es un descendiente de ese nodo.
3. En el preorden, “A” aparece antes que “T” porque es un ancestro de ese nodo.
4. Ninguna es correcta.

Parte 2

Si los siguientes son los recorridos en inorden y postorden de un árbol binario, dibuje el árbol que dio origen a esos recorridos.

- Inorden: R, A, T, B, J, S, M, C, W, Z, X, F
- Postorden: R, A, B, J, T, C, Z, W, F, X, M, S
- En el árbol resultante:
 1. “R” es la raíz, ya que aparece primero en ambos recorridos.
 2. “A” tiene que ser descendiente de “M”, ya que aparece antes en ambos recorridos.
 3. Existen varios árboles posibles que cumplen esa condición.
 4. Ninguna es correcta.
- En el árbol resultante:
 1. “B” es descendiente de “S”.
 2. “M” es ancestro de “C”.
 3. “F” es hermano de “W”.
 4. Todas son correctas.
- En el árbol resultante:
 1. El árbol tiene altura 4.
 2. El árbol tiene 6 hojas.
 3. “A” y “W” están al mismo nivel.
 4. Ninguna es correcta.

Parte 3

Dada la siguiente lista de claves, insértelas en un árbol binario de búsqueda, siguiendo el pseudo-código de inserción indicado en material de lectura.

T, Y, U, P, L, K, J, S, A, Z, X, C, V, N

- En el árbol resultante:
 1. El árbol tiene altura 7.
 2. “Y” es ancestro de “C”.

3. "X" es descendiente de "P".
 4. "S" es una hoja.
- En el árbol resultante:
 1. El árbol tiene cinco hojas.
 2. "V" es descendiente de "S".
 3. "K" es hermano de "J".
 4. "A" y "X" están al mismo nivel.
 - Seleccione la opción correcta:
 1. Para imprimir las claves en orden lexicográfico, basta con recorrer el árbol en preorden.
 2. Para imprimir las claves en orden lexicográfico, basta con recorrer el árbol en postorden.
 3. Para imprimir las claves en orden lexicográfico, basta con recorrer el árbol en inorden.
 4. Para imprimir las claves en orden lexicográfico, debe recorrerse el árbol por niveles.

Parte 4

1. El siguiente algoritmo aplicado a la raíz de un árbol binario, devuelve:

```

1: procedure MIFUNCION
2:   // retorna un tipo entero
3:    $A \leftarrow -1$ 
4:    $B \leftarrow -1$ 
5:   if this.hijoIzq  $\neq$  null then
6:      $A \leftarrow \text{this.hijoIzq.miFuncion}()$ 
7:   end if
8:   if this.hijoDer  $\neq$  null then
9:      $B \leftarrow \text{this.hijoDer.miFuncion}()$ 
10:  end if
11:  return  $\max(A, B) + 1$ 
12: end procedure

```

2. Analizar el orden del tiempo de ejecución del algoritmo.
3. Responder las siguientes preguntas:

a) ¿Qué calcula el algoritmo?

- 1) La altura del árbol.
- 2) El valor de la expresión aritmética contenida en el árbol.
- 3) El grado del árbol.
- 4) El tamaño del árbol.

b) El orden del tiempo de ejecución de este algoritmo es:

- 1) $O(n)$
- 2) $O(1)$
- 3) $O(\log_2(n))$
- 4) $O(n^2)$

Parte 5

Las siguientes listas son el resultado de aplicar un recorrido en preorden y en inorden a un árbol binario.

Preorden A, B, F, C, D, E, G

Inorden F, B, A, D, C, G, E

Seleccione la opción correcta

1. G es un descendiente de C.
2. B es un descendiente de F.
3. A es una hoja.
4. B es una hoja.

Parte 6

Dado un árbol binario de búsqueda inicialmente vacío, realice las siguientes operaciones:

1. Insertar las claves **12, 25, 14, 1, 33, 88, 45, 2, 7, 66, 5, 99**.
 - a) Escriba los recorridos en preorden, inorden y postorden.
 - b) Responda las preguntas presentadas en pantalla.
2. A partir del árbol final, elimine (dibujando paso a paso) las siguientes claves: **99, 2, 12, 33**.
 - a) Escriba los recorridos en preorden, inorden y postorden.
 - b) Después de las inserciones, el recorrido en preorden produce:
 - 1) 12, 1, 2, 5, 7, 25, 14, 33, 88, 45, 66, 99
 - 2) 12, 1, 2, 7, 5, 25, 14, 33, 45, 66, 88, 99
 - 3) 12, 1, 2, 5, 7, 25, 14, 33, 88, 45, 66, 99
 - 4) 12, 1, 2, 7, 5, 25, 14, 33, 88, 45, 66, 99
 - c) Después de las inserciones, el recorrido en inorden produce:
 - 1) 2, 1, 5, 7, 14, 12, 25, 33, 45, 66, 88, 99
 - 2) 1, 2, 5, 7, 12, 14, 25, 33, 45, 66, 88, 99

3) 1, 2, 5, 7, 12, 14, 25, 45, 33, 66, 88, 99

4) 1, 2, 7, 5, 12, 14, 25, 33, 45, 66, 88, 99

Parte 7

Sean:

- $p(x)$, la posición en preorden del nodo con identificador x .
- $s(x)$, la posición en postorden del nodo con identificador x .
- $i(x)$, la posición en inorden del nodo con identificador x .

Marque en el cuadro siguiente las posiciones que pueden ser ciertas simultáneamente.

	$i(n) < i(m)$	$s(n) < s(m)$	$p(n) < p(m)$
n es descendiente de m			
n está a la izquierda de m			
n está a la derecha de m			
n es ancestro de m			

Parte 8

Represente la siguiente expresión aritmética utilizando un árbol binario:

$$a - (b * (c + d / (f + g)) + h * (i - j * (k + l)))$$

Utilizando este árbol, dé un algoritmo para evaluar la expresión cuando las variables toman valores. Analice el orden del tiempo de ejecución de este algoritmo.

Ejercicio 2

Parte 1

El objetivo de este ejercicio es implementar varias operaciones del TDA `TDAArbolBinario`, a fin de aplicarlo más tarde a situaciones reales. Utilizar para esto los pseudocódigos publicados y las interfaces publicadas.

Paso 1: Todo el equipo

1. Crear las clases necesarias que implementen `TDAArbolBinario` y `TDAElemento` de acuerdo a las interfaces publicadas.
2. Sincronizar código entre todos.

Paso 2: En sub-equipos

Sub-Equipo A

1. Implementar y probar el algoritmo para insertar un nuevo dato en el árbol. El método debe implementar un contador que se incrementa con cada invocación al mismo y debe mostrar por consola el contenido final del contador.
2. En la clase `Main`, agregar funcionalidad que permita cargar las claves desde un archivo de texto `claves.txt`, en el cual habrá una clave por línea, y que escriba otro archivo

en que, en cada línea, escriba la clave insertada y el valor del contador (0 si no se pudo insertar).

3. Implementar un método que devuelva una string con el recorrido del árbol en preorden.

Sub-Equipo B

1. Implementar y probar el algoritmo para buscar un elemento en el árbol, dada una clave (comparable, string o entero).
2. En la clase **Main**, agregar funcionalidad que permita cargar las consultas desde un archivo de texto **consultas.txt**, en el cual habrá una clave por línea, y que escriba otro archivo en que, en cada línea, escriba la clave buscada y el valor del contador (con signo negativo si la clave no fue encontrada en el árbol).
3. Implementar un método que devuelva una string con el recorrido del árbol en postorden.
4. Implementar un método que devuelva una string con el recorrido del árbol en inorden.

Paso 3: Todo el equipo

1. Integrar los cambios en el repositorio.
2. Utilizando el programa realizado y los archivos que se liberarán, responder preguntas proyectadas en pantalla.

Parte 2

Agregar funcionalidades al TDArbolBinario.

Paso 1: En sub-equipos

Sub-Equipo A:

1. Implementar el algoritmo para obtener la altura del árbol.
2. Implementar el algoritmo para obtener la cantidad de hojas del árbol.
3. Probar los algoritmos insertando en el árbol varias claves y verificar en papel.

Sub-Equipo B:

- Implementar el algoritmo para obtener el tamaño del árbol.
- Implementar el algoritmo para, dado un criterio de búsqueda, indicar en qué nivel se encuentra en el árbol.
- Probar el algoritmo insertando en el árbol varias claves y verificar en papel.

Paso 2: Todo el equipo

- Integrar los cambios en el repositorio.
- Responder las preguntas presentadas.

Ejercicio 3

Parte 1

Dado un árbol binario de búsqueda inicialmente vacío, realice las siguientes operaciones:

1. Inserte las claves 12, 25, 14, 1, 33, 88, 45, 2, 7, 66, 5, 99.
2. Escriba los recorridos en preorden, inorden y postorden.
3. A partir del árbol final, y siguiendo el pseudocódigo de eliminación indicado, elimine (dibujando paso a paso) las siguientes claves:

99, 2, 12, 33

Parte 2

Paso 1: En sub-equipos

Sub-equipo A: Dado el pseudocódigo de eliminación en ABB, desarrolle los métodos correspondientes en JAVA. Tener visible el pseudocódigo de las slides.

Sub-equipo B: Desarrolle los casos de prueba para la funcionalidad de *eliminar* del TDA de árbol. En estos tests se debe:

1. Insertar elementos en el árbol con claves 12, 25, 14, 1, 33, 88, 45, 2, 7, 66, 5, 99.
2. Emitir por consola el recorrido en inorden.
3. Eliminar del árbol los elementos con las siguientes claves: 99, 15, 2, 12, 77, 33.
4. Emitir por consola los recorridos en inorden, preorden y postorden.
5. Verificar que la salida luego de efectuar cada eliminación es la esperada.

Paso 2: Todo el equipo

- Integrar los métodos desarrollados y los casos de prueba. Revisar que quede todo sincronizado en el repositorio.
- Ejecutar los test cases y verificar que los resultados son los mismos que los obtenidos en la Parte 1.